# Ψηφιακή Επεξεργασία Εικόνας
# Εργασία 2: Ανίχνευση ακμών και κύκλων

*Τζίνα Θεοδώρα (ΑΕΜ: 10715)*

*Μάιος 2025*

## 1 Εισαγωγή

## 2 FIR filters

2.1 *fir_conv.py*

## 3 Τελεστής ανίχνευσης ακμών Sobel

3.1 *sobel_edge.py*

## 4 Τελεστής ανίχνευσης ακμών Laplacian of Gaussian (LoG)

4.1 *log_edge.py*

## 5 Ανίχνευση κύκλων με τη μέθοδο Hough

5.1 *circ_hough.py*

## 6 Demo.py

6.1 Sobel edge detection

6.2 LoG edge detection

6.3 Hough transform

6.3.1 Using Sobel operator

6.3.2 Using LoG operator

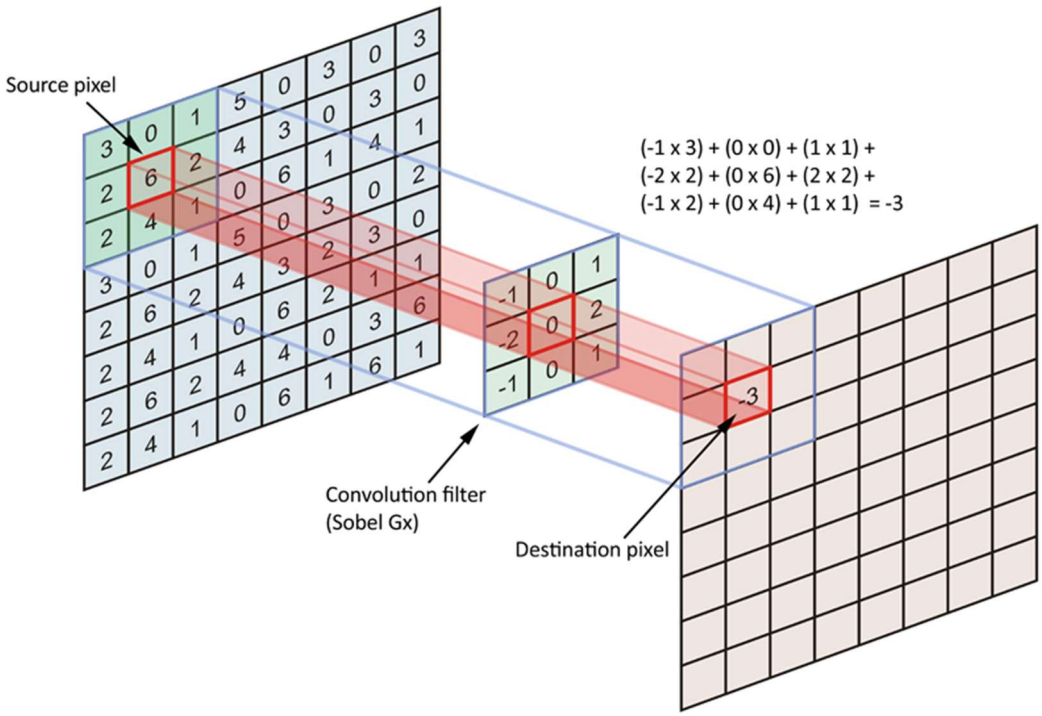
# 1 Εισαγωγή

Η παρούσα εργασία ασχολείται με την ανίχνευση ακμών και κύκλων σε μία εικόνα. Οι μέθοδοι ανίχνευσης βασίζονται στην πράξη της συνέλιξης FIR, η οποία αποτελεί και το πρώτο βήμα υλοποίησης της εργασίας. Δύο εναλλακτικοί μέθοδοι ανίχνευσης ακμών με τη βοήθεια συνέλιξης FIR, αποτελούν οι τελεστές Sobel και Laplacian of Gaussian (LoG). Οι τελεστές αυτοί, αφού επεξεργαστούν την εικόνα επιστρέφουν ως αποτέλεσμα μια δυαδική εικόνα, απαραίτητη για την εκτέλεση της μεθόδου Hough. Η μέθοδος αυτή αποτελεί το τελικό κομμάτι της εργασίας και είναι υπεύθυνη για την ανίχνευση κύκλων στην εικόνα. Με τα απαραίτητα βοηθητικά εργαλεία της python ανακτούμε την αρχική εικόνα εισόδου με σχεδιασμένα τα περιγράμματα των κύκλων.

# 2 FIR filters

Σε ένα Finite Impulse Resonse (FIR) σύστημα, η κρουστική απόκριση $h(n_1, n_2)$ έχει πεπερασμένη "περιοχή υποστήριξης", δηλαδή έχει μη μηδενικές τιμές σε μια πεπερασμένη περιοχή $W$. Η περιοχή αυτή, βρίσκεται μέσα σε ένα ορθογώνιο χωρίο $[N_{1,min}, \ldots, N_{1,max}] \times [N_{2,min}, \ldots, N_{2,max}]$ και επομένως η επίδραση του Γραμμικά χωρικά αναλλοίωτου συστήματος σε μία εικόνα εισόδου $f(n_1, n_2)$ μέσω της διαδικασίας συνέλιξης γράφεται ως εξής:

$$g(n_1, n_2) = \sum_{k_1=N_{1,min}}^{N_{1,max}} \sum_{k_2=N_{2,min}}^{N_{2,max}} h(k_1, k_2) f(n_1 - k_1, n_2 - k_2)$$



Οι τιμές της FIR κρουστικής απόκρισης συνήθως περιγράφονται από τη μορφή μίας συνελικτικής μάσκας, δηλαδή ενός πίνακα διάστασης $N_2 \times N_1$. Η σύμβαση αυτή είναι συνεπής με χρήση πινάκων για την αποθήκευση εικόνων $f(n_1, n_2)$.

### 2.1 *fir_conv.py*

Το αρχείο αυτό εμπεριέχει την ομώνυμη συνάρτηση, η οποία καλείται να υλοποιήσει την πράξη της συνέλιξης όπως παρουσιάστηκε παραπάνω. Αρχικά η συνάρτηση δέχεται σαν όρισμα την κανονικοποιημένη εικόνα *in_img_array* σε μορφή numpy array με τιμές στο διάστημα [0, 1] και τη μάσκα *h* επίσης σε μορφή numpy array. Υπολογίζει τις διαστάσεις της μάσκας, οι οποίες υποδεικνύουν την περιοχή εκτέλεσης του zero-padding στην αρχική εικόνα. Για τη διαδικασία αυτή χρησιμοποιείται η συνάρτηση numpy.pad(), δημιουργώντας ένα περίγραμμα μηδενικών στην αρχική εικόνα και αποθηκεύοντας το αποτέλεσμα στον βοηθητικό πίνακα *padded_img.* Αφού αρχικοποιήσει τον πίνακα εξόδου *out_img_array* με μηδενικά (black image), εκτελεί την πράξη της συνέλιξης μέσω ενός διπλού for loop. Εντός του loop, διατρέχονται ένα-ένα τα pixels της εικόνας και υπολογίζεται η γειτονική περιοχή του κάθε pixel, με την οποία πραγματοποιείται η συνέλιξη της μάσκας. Το αποτέλεσμα επιστρέφεται μέσω του numpy array *out_img_array.*

## 3 Τελεστής ανίχνευσης ακμών Sobel

Ο τελεστής Sobel είναι ένας από τους πιο διαδεδομένους τελεστές για την εξαγωγή ακμών. Υπολογίζει προσεγγιστικά τις παραγώγους πρώτης τάξης κατά τις κατευθύνσεις $x_1$ και $x_2$, μέσω της συνέλιξης της εικόνας με δύο μάσκες $3 \times 3$, μία για κάθε διεύθυνση:

$$G_{X1} = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_{x2} = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Οι μάσκες αυτές εφαρμόζονται στην εικόνα με συνέλιξη, και το αποτέλεσμα είναι δύο εικόνες $g_1(n_1, n_2)$ και $g_2(n_1, n_2)$ που περιέχουν διακριτές προσεγγίσεις των παραγώγων της $f(x_1, x_2)$ ως προς $x_1$ και $x_2$. Το συνολικό μέτρο της κλίσης σε κάθε σημείο υπολογίζεται με τη σχέση:

$$g(n_1, n_2) = \sqrt{g_1(n_1, n_2)^2 + g_2(n_1, n_2)^2}$$

Οι περιοχές της εικόνας όπου η τιμή της $g(n_1, n_2)$ είναι μεγάλη, αντιστοιχούν σε σημεία με έντονες μεταβολές φωτεινότητας, δηλαδή ακμές.

### 3.1 *sobel_edge.py*

Το αρχείο αυτό εμπεριέχει την ομώνυμη συνάρτηση, η οποία καλείται να ανιχνεύσει τις ακμές μιας εικόνας σύμφωνα με την προαναφερθείσα θεωρία. Αρχικά η συνάρτηση ορίζει δύο numpy array πίνακες με τις τιμές των $3 \times 3$ παραπάνω μασκών. Εκτελεί την συνέλιξη αυτών με την εικόνα εισόδου *in_img_array*, καλώντας την ήδη υλοποιημένη συνάρτηση *fir_conv,* υπολογίζοντας τις παραγώγους πρώτης τάξης κατά τις κατευθύνσεις $x_1$ και $x_2$. Έπειτα, υπολογίζει το μέτρο της κλίσης κάθε σημείου με τη βοήθεια της numpy.sqrt, των αποτελεσμάτων των συνελίξεων και του τύπου της θεωρίας. Αφού αρχικοποιήσει την *out_img_array* με μηδενικά (black image), εκτελεί ένα διπλό for loop, διατρέχοντας ένα-ένα τα pixels της εικόνας. Στόχος είναι να εντοπίσει τα pixels όπου το μέτρο της κλίσης τους ξεπερνάει την τιμή ενός κατωφλίου, δηλαδή του ορίσματος *thres* της συνάρτησης. Τα pixels αυτά αντικαθίστανται από την τιμή 1, φέρνοντας ως αποτέλεσμα μια binary εικόνα, η οποία επιστρέφεται από τη συνάρτηση ως *out_image_array*, μορφής numpy array.

# 4 Τελεστής ανίχνευσης ακμών Laplacian of Gaussian (LoG)

Ο τελεστής Laplacian of Gaussian (LoG) χρησιμοποιείται επίσης για την ανίχνευση ακμών μέσω όμως των δεύτερων παραγώγων αυτή τη φορά. Στόχος είναι ο εντοπισμός των περιοχών όπου η δεύτερη παράγωγος της έντασης της εικόνας μεταβάλλεται απότομα, δηλαδή εκεί που υπάρχουν ακμές. Η μέθοδος αυτή πραγματοποιεί πρώτα εξομάλυνση της εικόνας με έναν διδιάστατο Γκαουσιανό πυρήνα, ώστε να μειωθεί ο θόρυβος, και στη συνέχεια υπολογίζει τη Λαπλασιανή της εικόνας, δηλαδή το άθροισμα των δεύτερων μερικών παραγώγων. Η διαδικασία αυτή, συνοψίζεται σε έναν ενιαίο τελεστή, που προκύπτει από την εφαρμογή της Λαπλασιανής στον Γκαουσιανό πυρήνα. Ο τελικός πυρήνας συνελίξεως (LoG kernel) που μας αφορά, έχει τη μορφή:

$$LoG(x_1 x_2) = -\frac{1}{\pi\sigma^4}\left(1 - \frac{{x_1}^2 + {x_2}^2}{2\sigma^2}\right) e^{-\frac{{x_1}^2+{x_2}^2}{2\sigma^2}}$$

Στην πράξη η διαδικασία υλοποιείται με συνέλιξη με έναν πίνακα συντελεστών (μια μάσκα) διαστάσεων $L_1 \times L_2$, όπου οι τιμές του LoG υπολογίζονται σε ένα δειγματοληπτημένο πλέγμα γύρω από το κέντρο (π.χ. $x_1, x_2 \in \{-k, \ldots, 0, \ldots, +k\}$, με $L_1 = L_2 = 2k + 1$). Η συνέλιξη της εικόνας με τη μάσκα αυτή, οδηγεί σε μια νέα εικόνα, στην οποία οι ακμές ανιχνεύονται εύκολα, εντοπίζοντας τα μηδενικά σημεία (zero-crossings).

Ο τελεστής αυτός, συνεπώς, είναι ιδανικός για εφαρμογές όπου απαιτείται μείωση του θορύβου και αυξημένη ευαισθησία στις λεπτομέρειες.

### 4.1 *log_edge.py*

Το αρχείο αυτό εμπεριέχει την ομώνυμη συνάρτηση, η οποία είναι επίσης υπεύθυνη για τον εντοπισμό ακμών στην εικόνα εισόδου *in_img_array.* Ως όρισμα της συνάρτησης έχει προστεθεί η μεταβλητή *thres,* η οποία λειτουργεί ως κατώφλι κατά τη διαδικασία του zero-crossings. Αρχικά αρχικοποιείται η μάσκα με τις τιμές του τελεστή LoG, η οποία θα εφαρμοστεί στη συνέλιξη με την εικόνα. Επιλέγουμε μια μάσκα (μορφής "Mexican hat") $5 \times 5$:

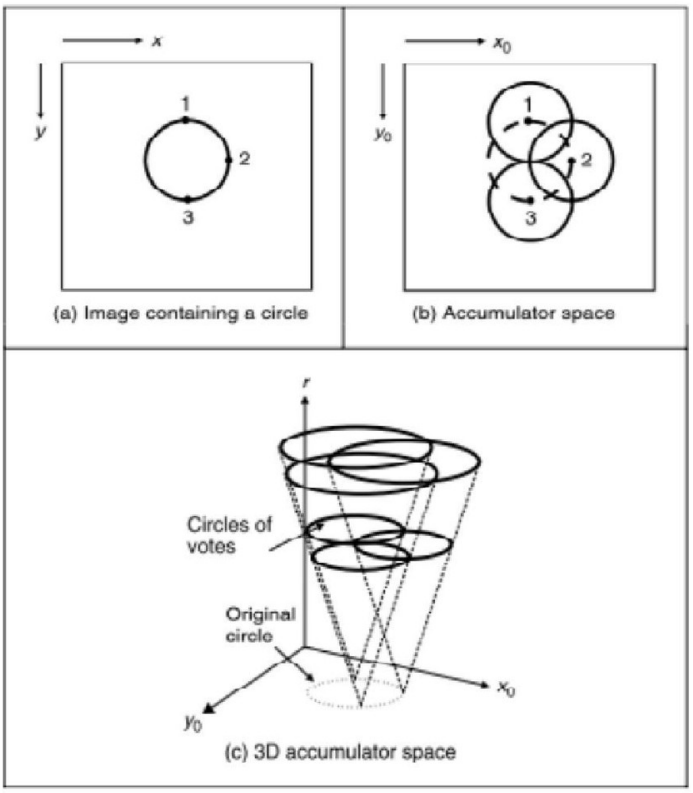
$$h = \begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & -2 & -1 & 0 \\ -1 & -2 & 16 & -2 & -1 \\ 0 & -1 & -2 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix}$$

Η συνάρτηση καλεί την ήδη υλοποιημένη συνάρτηση *fir_conv,* η οποία πραγματοποιεί την παραπάνω συνέλιξη και αποθηκεύσει το αποτέλεσμα στον βοηθητικό πίνακα *filtered_img.* Αφού αρχικοποιήσει τον πίνακα εξόδου *out_img_array* με μηδενικά (black image), χρησιμοποιεί ένα διπλό for loop όπου ελέγχει τη γειτονιά του κάθε pixel του *filtered_img* και αναζητεί σημεία μηδενικής διέλευσης (zero-crossings). Προσέχουμε το loop να αποφεύγει τα ακριανά πάνω-κάτω και δεξιά-αριστερά pixels σε κάθε επανάληψη για να παραμείνουμε εντός του πλαισίου της εικόνας. Κατά τη διαδικασία του zero-crossings ελέγχουμε τόσο τα πάνω-κάτω και αριστερά-δεξιά γειτονικά pixels, όσο και τα διαγωνίως. Όπου εντοπιστεί αλλαγή προσήμου, δηλαδή μηδενική διέλευση, αλλά και διαφορά μεγαλύτερη του *thres* ανάμεσα στα ετερόσημα στοιχεία, σημαίνει ότι υπάρχει απότομη αλλαγή φωτεινότητας και άρα ακμή. Επομένως στη θέση αυτή αντίστοιχα του *out_img_array* τοποθετείται η τιμή 1 και τελικά η συνάρτηση επιστρέφει τη ζητούμενη binary εικόνα σε μορφή numpy array.

## 5 Ανίχνευση κύκλων με τη μέθοδο Hough

Η μέθοδος Hough γενικά ανιχνεύει παραμετρικές καμπύλες σε δυαδικές (binary) εικόνες, όπου τα σημεία της καμπύλης, δηλαδή οι ακμές, έχουν άσπρο χρώμα (τιμή 1) και το background είναι μαύρο (τιμή 0). Η μέθοδος δύναται να ανιχνεύσει τις καμπύλες ακόμη κι άν σε ένα μεγάλο μέρος της υπάρχουν κενά, μαύρα σημεία ή στο background υπάρχουν περιττές ακμές, άσπρα σημεία. Στις παραμετρικές καμπύλες που ανιχνεύονται με τη μέθοδο Hough συμπεριλαμβάνονται και οι

κύκλοι, τους οποίους καλούμαστε να εντοπίσουμε σε αυτή την εργασία. Οι κύκλοι προσδιορίζονται από 3 παραμέτρους: δύο για το κέντρο $(a, b) \in [0,N_1-1] \times [0,N_2-1]$ και μία για την ακτίνα $\rho \in [0,R_{max}]$. Συνεπώς ο πίνακας ψηφοφορίας του Hough είναι 3D πίνακας διαστάσεων $K \times L \times M$ όπου οι ακέραιοι $K, L, M$ αντιστοιχουν στο πλήθος των υποδιαστημάτων στα οποία διαμερίζονται τα παραπάνω διαστήματα τιμών των $a, b, \rho$. Κάθε κελί του πίνακα συσσώρευσης ψήφων παραμετροποιεί έναν κύκλο.




## 5.1 *circ_hough.py*

Το αρχείο αυτό εμπεριέχει την ομώνυμη συνάρτηση, η οποία καλείται να ανιχνεύσει τους κύκλους της εικόνας *in_img_array,* η οποία αυτή την φορά δεν θα αποτελεί την αρχική εικόνα εισόδου, αλλά μια binary εικόνα ακμών προερχόμενη από τη συνάρτηση *sobel_edge* ή *log_edge.* Η συνάρτηση *circ_hough* έχει επίσης ως ορίσματα: τη μεταβλητή *R_max,* που υποδεικνύει το μέγιστο του εύρους τιμών της ακτίνας του κύκλου, τον numpy array πίνακα *dim* που εμπεριέχει τις $K \times L \times M$ διαστάσεις του πίνακα συσσώρευσης, και τη μεταβλητή *V_min* που δηλώνει το ελάχιστο πλήθος ψήφων ενός κελιού του πίνακα συσσώρευσης προκειμένου να θεωρηθεί ότι αντιστοιχεί σε κύκλο. Επίσης προστέθηκε το όρισμα *R_min,* για διευκόλυνση της διαδικασίας αναζήτησης κύκλων. Αρχικά, αρχικοποιείται ο πίνακας Hough με τις δοθείσες διαστάσεις, ο πίνακας *R_vals* που εμπεριέχει τις τιμές υποψήφιας ακτίνας και το βήμα διαχωρισμού κάθε παραμέτρου. Μέσω ενός τριπλού for loop διατρέχονται

ένα-ένα τα μη μηδενικά σημεία της δυαδικής εικόνας, δηλαδή οι ακμές της, οι τιμές *R_vals* και οι τιμές *theta* οι οποίες παραμετροποιούν μια γωνία θ που βοηθά στον υπολογισμό των πιθανών κέντρων μετέπειτα. Για κάθε τιμή μιας υποψήφιας ακτίνας και μιας γωνίας, υπολογίζονται οι συντεταγμένες *(a,b)* του υποψήφιου κέντρου. Έπειτα γίνεται παραμετροποίηση των τριών αυτών τιμών και εισαγωγή τους στο αντίστοιχο κελί του πίνακα συσσώρευσης. Αφού ελεγχθούν όλα τα pixels, εκτελείται η αντίστροφη διαδικασία υπολογισμού των ακριβών παραμέτρων του κύκλου, για τα κελιά εκείνα τα οποία απαριθμούν περισσότερες ψήφους από την τιμή του *V_min.* Αφού υπολογιστούν τα κέντρα *(a,b)* και οι ακτίνες *r* που ικανοποιούν τις προϋποθέσεις ύπαρξης κύκλου γίνεται εισαγωγή τους στους αντίστοιχους numpy arrays, *centers* και *radii,* οι οποίοι αποτελούν τα αποτελέσματα της συνάρτησης *circ_hough.*

# 6 Demo.py

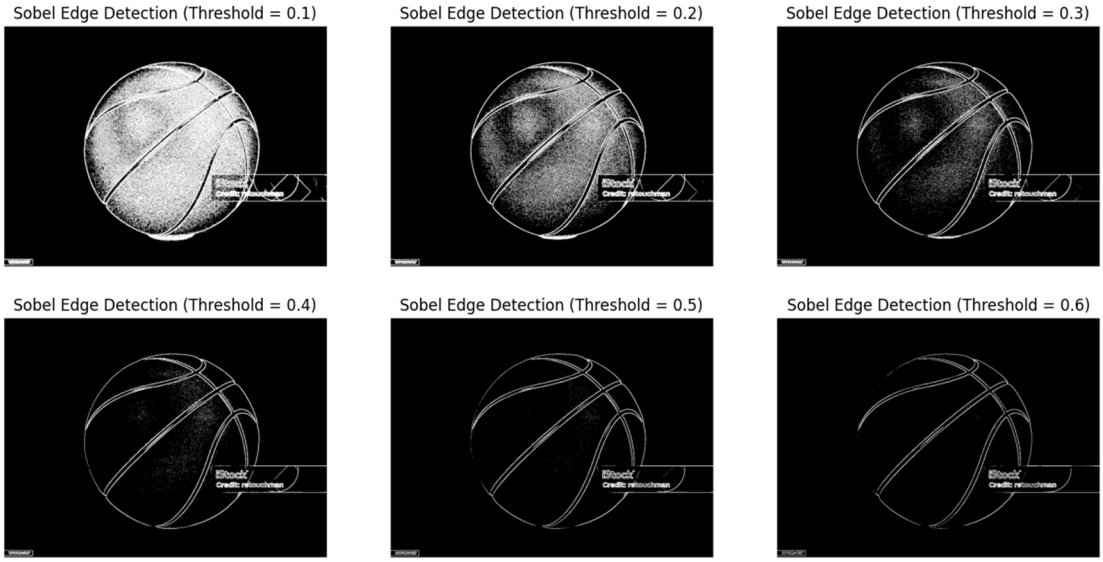
Από το τελικό αυτό αρχείο προκύπτουν οι ζητούμενες απεικονίσεις των αποτελεσμάτων των παραπάνω συναρτήσεων. Αρχικά πραγματοποιείται η ανάγνωση και μετατροπή της εικόνας σε ασπρόμαυρη μορφή μέσω του πακέτου pillow. Για την εφαρμογή της στις υλοποιημένες συναρτήσεις απαιτείται η μετατροπή της σε μορφή κανονικοποιημένου πίνακα με τιμές στο διάστημα [0, 1]. Για την εκτύπωση τόσο της αρχικής εικόνας εισόδου, όσο και των επεξεργασμένων που θα ακολουθήσουν χρησιμοποιείται η βιβλιοθήκη matplotlib.

Η αρχική εικόνα που καλούμαστε να επεξεργαστούμε πριν και μετά την εισαγωγή της στο python αρχείο.

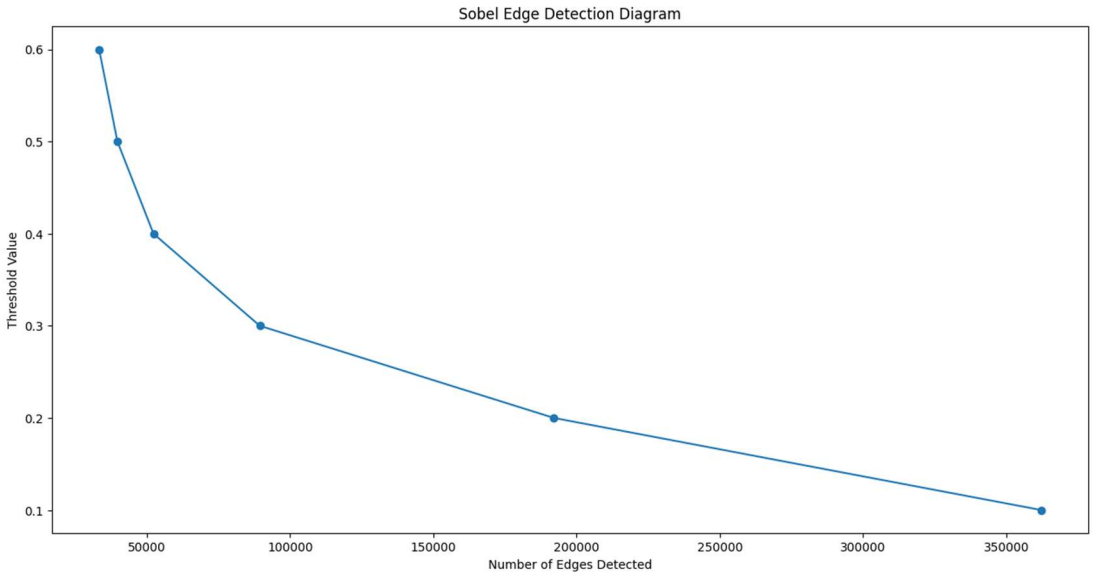
## 6.1 Sobel edge detection

Για την ανίχνευση ακμών με τον τελεστή Sobel, επιλέγεται μια ποικιλία τιμών κατωφλίου στο εύρος [0,1 – 0,6] με σκοπό την εύρεση του

καλύτερου δυνατού αποτελέσματος. Καλείται η συνάρτηση *sobel_edge* για τις διάφορες αυτές τιμές και προκύπτουν τα εξής αποτελέσματα:



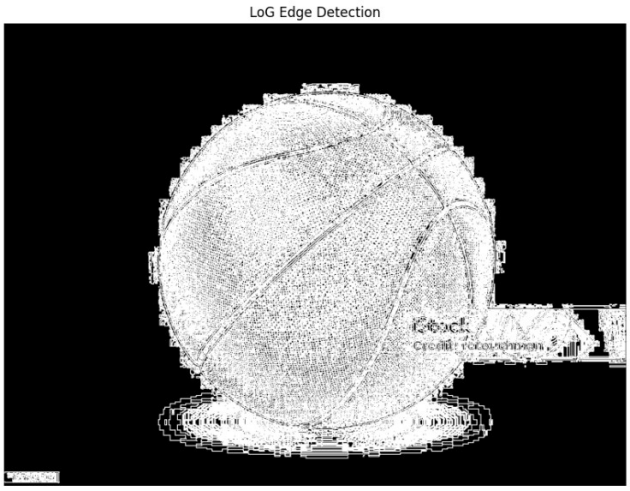

Παρατηρούμε ότι όσο μεγαλύτερη είναι η τιμή του κατωφλίου (threshold) τόσο λιγότερες ακμές εντοπίζονται. Ενώ όσο μειώνεται η τιμή του κατωφλίου, το πλήθος των ακμών αυξάνεται εκθετικά. Η παρατήρηση αυτή επιβεβαιώνεται και από την απεικόνιση του διαγράμματος "Threshold value – Number of Edges Detected":



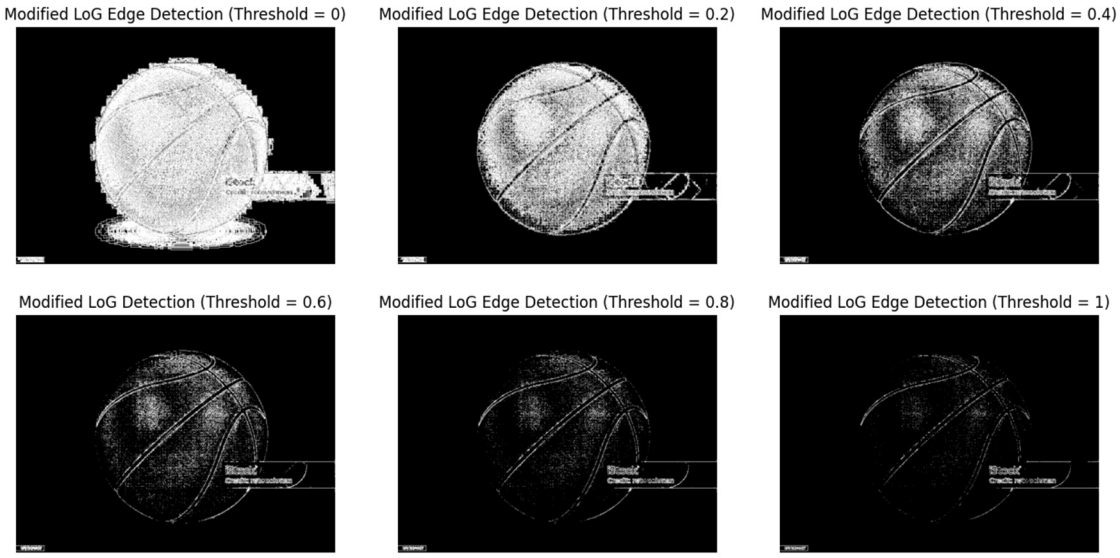

Εφόσον στόχος μας αποτελεί μετέπειτα η χρήση της δυαδικής εικόνας που προκύπτει από την συνάρτηση *sobel_edge,* στη συνάρτηση *circ_hough* που είναι υπεύθυνη για τον εντοπισμό κύκλων, συμπεραίνουμε ότι βέλτιστη επιλογή αποτελεί η τιμή του threshold=0,5. Με το κατώφλι αυτό περιορίζονται οι περιττές ακμές, αλλά παραμένει αρκετό το πλήθος των ακμών στο περίγραμμα του κύκλου προκειμένου να καταφέρει η *circ_hough* να το εντοπίσει.

## 6.2 LoG edge detection

Για την ανίχνευση ακμών με τον τελεστή Laplacian of Gaussian (LoG), δεν υπήρχε αρχικά κάποιο όρισμα μεταβολής του αλγορίθμου και χρησιμοποιήθηκε και προκαθορισμένη μάσκα κατά την υλοποίησή του. Επομένως η κλήση της συνάρτησης *log_edge,* έδινε αρχικά το εξής αποτέλεσμα:




Παρατηρούμε ότι, δεδομένης της προκαθορισμένης μάσκας που χρησιμοποιήθηκε, το πλήθος των ακμών που εντοπίστηκαν είναι ιδιαίτερα αυξημένο, γεγονός που επιβεβαιώνεται και από τον μεγάλο χρόνο εκτέλεσης του αλγορίθμου. Η πλειονότητα των ακμών είναι περιττές κατά τη διαδικασία εντοπισμού κύκλων και μια τέτοια εφαρμογή είναι ιδιαίτερα κοστοβόρα και χρονοβόρα. Για τον λόγο αυτό, όπως αναφέρθηκε και στην ενότητα 4.1, επιλέξαμε να προσθέσουμε ένα νέο όρισμα στη συνάρτηση, τη μεταβλητή *thres.* Η μεταβλητή αυτή, λειτουργεί ως κατώφλι κατά την εκτέλεση του zero-crossings, με σκοπό να περιοριστούν όσο το δυνατόν οι περιττές ακμές. Για διάφορες τιμές του *thres* προκύπτουν τα εξής αποτελέσματα:



Παρατηρούμε ότι πράγματι με τη βοήθεια του κατωφλίου περιορίζονται αρκετά οι περιττές ακμές στο κέντρο, αλλά ταυτόχρονα περιορίζονται αισθητά και οι ακμές στο περίγραμμα του κύκλου. Συμπεραίνουμε ότι βέλτιστη επιλογή για τη μετέπειτα εκτέλεση της *circ_hough,* αποτελεί η τιμή του threshold=0,4. Έτσι υπάρχει ακόμα μεγάλη πιθανότητα εντοπισμού του κύκλου και ταυτόχρονα ελαττώνονται όσο το δυνατόν οι εσωτερικές ακμές.

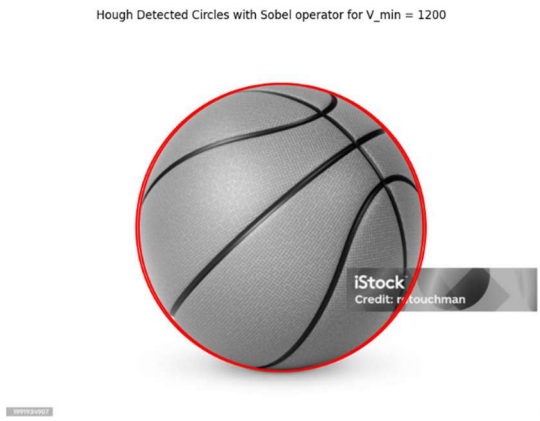
Η υπεροχή του τελεστή Sobel από τον τελεστή LoG τόσο σε απεικόνιση όσο και λειτουργικότητα είναι εμφανής, καθώς με τον πρώτο επιτυγχάνεται η καθαρή διατήρηση του περιγράμματος του κύκλου πολύ εύκολα. Για τον λόγο αυτό, αναμένονται και τα ανάλογα αποτελέσματα στην πορεία, κατά την εκτέλεση της διαδικασίας εντοπισμού κύκλων.

## 6.3 Hough transform

Για την ανίχνευση κύκλων με τη μέθοδο Hough, χρησιμοποιείται η αντίστοιχη συνάρτηση *circ_hough,* η οποία απαιτεί ως όρισμα μια binary εικόνα. Για την άντληση μιας τέτοιας εικόνας καλούνται οι βοηθητικές συναρτήσεις ανίχνευσης ακμών που προαναφέρθηκαν ήδη, *sober_edge* και *log_edge.* Για την απεικόνιση των αποτελεσμάτων χρησιμοποιήθηκε η συνάρτηση matplotlib.patches.Circle, αφού πρώτα πραγματοποιήθηκε μείωση της ανάλυσης της εικόνας στα ¾ της αρχικής για επίσπευση των αποτελεσμάτων χωρίς υπερβολική παραμόρφωσή της.

### 6.3.1 Using Sobel operator

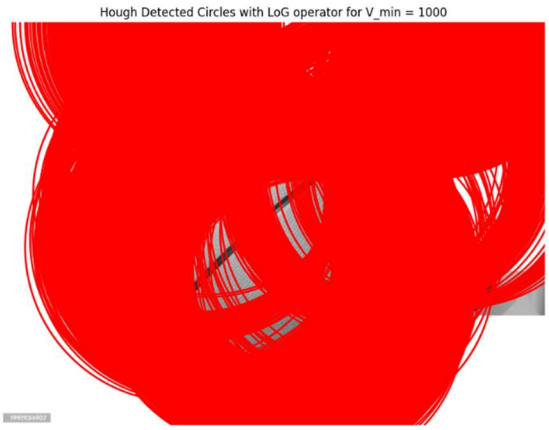
Για τις διάφορες τιμές του *V_min* μέσω της Sobel edge detection, επιλέγονται οι τιμές 600, 800, 1200, 1400, 1600. Για τις υπόλοιπες σταθερές παραμέτρους μετά από πειραματισμό επιλέχθηκαν οι τιμές: *R_max = 320, R_min = 280, dim = [200, 150, 5].*

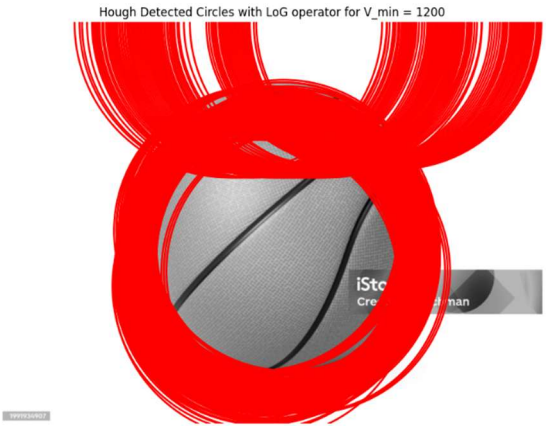


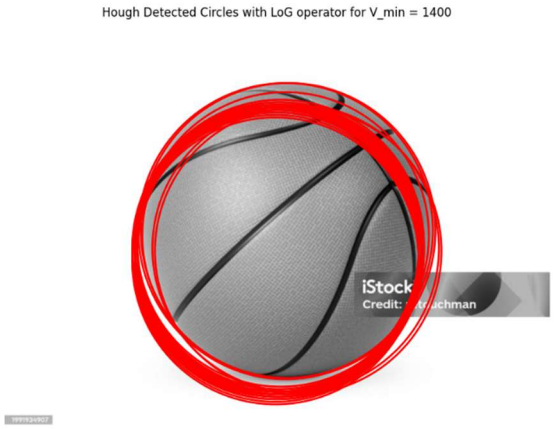
Παρατηρούμε ότι βέλτιστα αποτελέσματα παίρνουμε για την τιμή *V_min = 1400.*

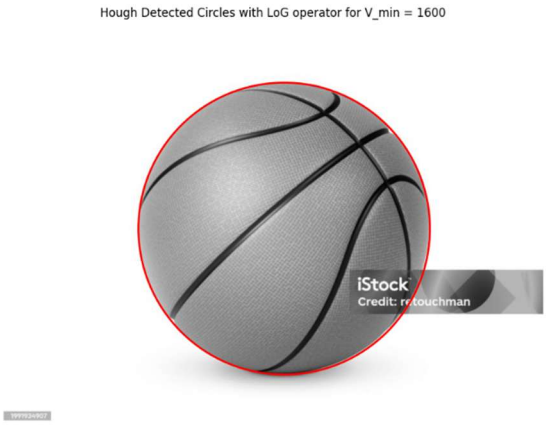
### 6.3.2 Using LoG operator

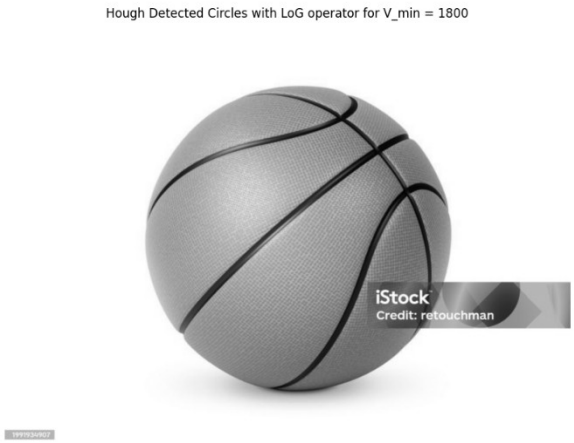
Για τις διάφορες τιμές του *V_min* μέσω της LoG edge detection, επιλέγονται οι τιμές 1000, 1200, 1400, 1600, 1800. Για τις υπόλοιπες σταθερές παραμέτρους μετά από πειραματισμό επιλέχθηκαν πάλι οι τιμές: *R_max = 320, R_min = 280, dim = [200, 150, 5].*






Παρατηρούμε ότι βέλτιστα αποτελέσματα παίρνουμε για την τιμή *V_min = 1600.*

Συμπεραίνουμε ότι για την ανίχνευση του κυκλικού περιγράμματος της μπάλας απαιτείται μεγαλύτερη τιμή *V_min* μέσω του τελεστή *LoG* από ότι μέσω του *Sobel.* Αυτό οφείλεται όπως αναφέρθηκε και νωρίτερα στο ότι η ανίχνευση ακμών *LoG* δεν είναι τόσο αποδοτική στην ταυτόχρονη μείωση θορύβου και διατήρηση των ακμών του περιγράμματος, όσο η ανίχνευση ακμών *Sobel.*